

ADR-0001 — Vía de autenticación RLS + plano de portabilidad WORM/OCC

Estado: Aprobado con condiciones (ver veredicto §5)

Fecha: 2026-08-11

Autor: 002_ARQUITECTO_DE_SOFTWARE, en respuesta a escalada de 005_INGENIERO_BACKEND (bloqueo estructural detectado antes de tocar la base de datos)

Alcance: proyecto raíz Antigravity JS (rama master), backend Node/Express + Supabase REST/PostgREST

1. Contexto verificado (no se asume nada de la escalada sin releer el código)

005_INGENIERO_BACKEND reportó 2 bloqueos. Antes de fallar el veredicto se relejó el código de los 3 proyectos involucrados — el hallazgo cambia la naturaleza de la decisión:

1.1 Fuga de perímetro (Formulador, proyecto raíz)

src/modules/formulador/supabaseClient.js:27-38,51-68 — sbFetch() degrada siempre a SUPABASE_SERVICE_KEY (bypassa RLS) porque PostgREST rechaza el JWT de Firebase con 401 (Third-Party Auth no configurado). El aislamiento real hoy depende de WHERE tenant_id = p_tenant_id explícito en cada RPC + assertValidTenant() (guardrail Node que aborta antes de tocar Postgres si el tenant es inválido). **No es "cero control" — es un control a nivel de query/aplicación, no de rol de Postgres.**

1.2 Los subsistemas WORM/OCC de los proyectos hermanos NO son tan portables como parecían — hallazgo nuevo

Se leyeron íntegros los 2 archivos que 005 propuso portar:

- proyectos/Proy_05_SIG/app/migrations/008_sprint6_worm.sql — bloquea UPDATE / DELETE con triggers RAISE EXCEPTION. **Esta parte es pura DDL de Postgres — no depende de cómo se conecta la app (REST o pg.Pool). Portable tal cual.**
- proyectos/Proy_03_RadarFondos/backend/migrations/009_project_version_hashes.sql — misma técnica de trigger append-only (portable igual), **pero sus políticas RLS usan 2 funciones distintas** (010_rls_complete_audit.sql:30-41):
- current_tenant_uuid() → lee current_setting('app.tenant_id', TRUE) — un **GUC de sesión custom**. Esto **exige una conexión Postgres persistente** (pg.Pool) que ejecute SET app.tenant_id = '<uuid>' antes de cada query en la misma transacción. **PostgREST (REST stateless) no puede setear esto** — cada request HTTP es una conexión nueva sin estado compartido.
- current_auth_uid() → llama auth.uid(), función nativa de Supabase que sí funciona sobre REST, **pero solo si PostgREST puede decodificar el JWT** — exactamente la misma precondition de Third-Party Auth que bloquea al proyecto raíz hoy.

Conclusión de 1.2, la pieza que cambia el veredicto: el patrón RLS del proyecto hermano no es una alternativa que evite configurar Third-Party Auth — su rama auth.uid() tiene la *misma* dependencia externa, y su rama app.tenant_id requiere reintroducir pg.Pool, que este proyecto **retiró deliberadamente de package.json** en una sesión anterior (docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md §4, decisión ya documentada). No hay una tercera vía gratis.

1.3 Hallazgo cruzado no pedido, pero relevante para el veredicto — la RLS del propio proyecto hermano también degrada a SERVICE_KEY

proyectos/Proy_03_RadarFondos/backend/config/database.config.js:1-50 documenta 3 capas: **Capa 1** (pg.Pool, la que hace funcionar current_tenant_uuid()) solo se activa si "el usuario habilita el pooler en Supabase dashboard" — acción externa, no garantizada; si no está activa, cae a **Capa 2** (REST + SUPABASE_SERVICE_KEY, bypass total de RLS) — **el mismo patrón que el hallazgo 1.1 del proyecto raíz**, no uno "más seguro". El propio archivo trae además una nota de que una credencial service_role real quedó commiteada ahí antes (línea 30-31) — dado que ambos proyectos comparten instancia Supabase (memoria ya registrada del usuario), **recomiendo verificar si es la misma credencial ya marcada crítica en docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md §6.4** antes de dar por cerrada esa revocación — no se puede confirmar desde disco, requiere que el humano lo verifique en el dashboard.

2. Decisión 1 — vía de autenticación

Veredicto: Opción A (Third-Party Auth) como objetivo, Opción B (encapsulamiento por RPC) se mantiene activa como control compensatorio durante la transición — no son mutuamente excluyentes, es una secuencia, no una elección binaria.

Razones:

1. La opción "raíz sigue con RPC + guardrail Node" (formalizar B como definitivo) deja el sistema **sin RLS real de Postgres** — el guardrail vive en Node, no en la base de datos; un bug futuro en una sola RPC nueva puede saltárselo sin que nada en Postgres lo impida. No es aceptable como estado final para un sistema que va a manejar datos WORM/financieros.
2. La opción "reintroducir `pg.Pool` para usar `app.tenant_id`" (calcar el patrón crudo de `Proy_03`) revierte una decisión arquitectónica ya tomada y documentada, y —según 1.3— **ni siquiera resuelve el problema de fondo** en el proyecto que se estaría copiando (su propia Capa 1 depende de una acción externa no garantizada).
3. Third-Party Auth es una configuración **una sola vez, en el dashboard de Supabase** (mapear Firebase como proveedor JWT reconocido) — sin reintroducir dependencias de conexión con estado, sin tocar `package.json`, compatible con el REST-only ya elegido. Una vez activo, `auth.uid()` funciona de verdad sobre PostgREST y las políticas RLS de ambos hermanos son portables sin reescribir su mecanismo de fondo.

Mientras Third-Party Auth no esté configurado (acción humana, fuera de alcance de cualquier agente): `assertValidTenant()` + el filtro `WHERE tenant_id` explícito en cada RPC siguen siendo el control primario, documentados como tal (no como "temporal sin dueño") — `005` no debe intentar removerlos ni debe bloquear el fallback a `SERVICE_KEY` mientras esta sea la única vía funcional del módulo Formulador.

3. Decisión 2 — plano de portabilidad WORM/OCC

Veredicto: portar la parte de triggers (bloqueo de UPDATE/DELETE) ahora, sin esperar Third-Party Auth. Portar las políticas RLS de los ledgers en una segunda migración, condicionada a que Decisión 1 ya esté resuelta.

Secuencia exacta que `005` debe seguir (esto es la especificación — la implementación SQL/migración queda para `005`, no para este documento):

1. **Migración A (aplicable ya):** tabla `project_version_hashes` (calcada de `009_project_version_hashes.sql`) + sus 2 triggers append-only (`trg_pvh_no_update`, `trg_pvh_no_delete`) — **sin las políticas RLS todavía**, o con una política provisional `USING (true)` explícitamente marcada como temporal en un comentario SQL, para no bloquear el desarrollo mientras Decisión 1 no esté resuelta. El sellado WORM de hallazgos/auditorías (patrón `008_sprint6_worm.sql`) se aplica igual — es DDL puro, no depende de RLS.
2. **Migración B (bloqueada hasta Third-Party Auth activo):** políticas RLS reales sobre ambas tablas, usando **únicamente** la rama `auth.uid()` (`current_auth_uid()` o equivalente) — **no portar la rama `app.tenant_id` /GUC de sesión**, es incompatible con la arquitectura REST-only de este proyecto y no debe reaparecer aquí.
3. **OCC (`version_hash` en PUT/PATCH):** el mecanismo de comparación de versión (HTTP 409 si difiere) es lógica de aplicación (Node), no depende de RLS — `005` puede implementarlo en paralelo a la Migración A, sin esperar Decisión 1.
4. **Shadow Ledger de violaciones** (Fase 4.2 del mandato de `005`): tabla nueva `security_violations_ledger` (append-only, mismo patrón de triggers que `project_version_hashes`) — mismo criterio: aplicable ya, sin RLS condicionada hasta Migración B.

4. Actualización del gate

`005_INGENIERO_BACKEND` queda desbloqueado para ejecutar **Migración A** y el mecanismo OCC de aplicación, bajo este ADR — no requiere un segundo veredicto de `002` para esos dos ítems específicos. **Migración B (RLS real) sigue bloqueada** hasta que el usuario confirme en el dashboard de Supabase que Third-Party Auth (Firebase) está configurado — ninguna cantidad de código puede sustituir esa confirmación.

5. Veredicto estructurado

```
{
  "aprobado": true,
  "condiciones": [
    "005 puede aplicar Migración A (triggers WORM + tabla project_version_hashes sin RLS real, o con policy USING(true) marcada exp
    "005 NO puede aplicar Migración B (políticas RLS reales) hasta que el usuario confirme Third-Party Auth activo en el dashboard
    "005 no debe remover ni bloquear el fallback a SERVICE_KEY en supabaseClient.js mientras sea la única vía funcional del módulo
    "005 no debe reintroducir el paquete pg ni SET LOCAL app.tenant_id en este proyecto – la rama app.tenant_id de las políticas de
    "pendiente humano fuera de este ADR: verificar si la credencial service_role comprometida en proyectos/Proy_03_RadarFondos/back
```

```
]
}
```

